



Tarefa Prática – Mini-Blockchain Simétrica com Autenticação de Usuário

1. Contexto

Você deve implementar uma **mini-blockchain multiusuário**, onde cada usuário pode registrar **transações** de forma segura.

O sistema deve garantir:

- **Confidencialidade:** dados de cada usuário são criptografados individualmente com **AES-GCM**.
- **Autenticação forte:** cada usuário deve fornecer **senha + TOTP** para adicionar blocos.
- **Integridade da blockchain:** cada bloco depende do hash do bloco anterior, formando uma cadeia imutável.
- **Multiusuário:** vários usuários podem adicionar blocos, mas cada um só acessa seus próprios dados.

O código pode ser feito em qualquer linguagem de programação que possua biblioteca criptográfica compatível com a funcionalidade necessária. No moodle, existe o arquivo compactado exemplosFIPS2.zip com códigos em Java que podem ser usados para o desenvolvimento da tarefa.

2. Requisitos Funcionais

2.1 Cadastro e Login de Usuário

- Usuário se cadastra com:
 - username
 - senha
- Senha derivada com **PBKDF2** ou **scrypt**
- TOTP - armazenar chave do TOTP para autenticação de dois fatores. O cálculo $TOTP = HMAC(chave, time)$ é feito com métodos prontos das bibliotecas criptográficas.
- Login requer:
 - senha correta
 - TOTP válido
- Após login, o sistema gera uma **chave de sessão segura** para cifragem/decifragem dos blocos do usuário

2.2 Registro de Blocos

- Usuário autenticado pode criar um bloco com dados arbitrários (ex.: transação, identidade, mensagem)
- Cada bloco contém:
 - dados → criptografado com **AES-GCM** usando chave derivada do usuário ou chave de sessão
 - iv único para AES-GCM
 - timestamp de criação
 - hash_prev → hash do bloco anterior da blockchain
 - owner → ID do usuário (username)
- Cada usuário só pode registrar **blocos pertencentes a si mesmo**

2.3 Leitura da Blockchain

- Usuário pode listar todos os blocos, mas **só decifra seus próprios dados**
- Para cada bloco, verificar:
 - integridade do hash_prev

- validade do AES-GCM (detecção de alterações)

2.4 Segurança

- **AES-GCM** garante confidencialidade e integridade dos dados
- **IV único** por bloco
- **Derivação de chave** com PBKDF2 ou scrypt
- **TOTP** obrigatório para adicionar blocos
- Sistema deve detectar tentativas de acesso ou alteração não autorizada

3. Funcionalidades Extras (opcionais)

- Permitir múltiplas chaves por usuário para diferentes tipos de dados
- Permitir expiração de blocos ou atualização de dados de identidade
- Logs de acesso e tentativa de manipulação de blocos
- Validação automática da cadeia a cada novo bloco

4. Funções do Menu e Casos de Teste Sugeridos

Scripts ou funções de teste (via menu simples, em modo texto):

1. cadastro/login
2. adição de blocos de múltiplos usuários
3. leitura e validação de blockchain por usuário
4. teste de manipulação de blocos para verificação de integridade

Autenticação

1. Login correto + TOTP válido → sucesso
2. Login com TOTP inválido → falha
3. Login com senha incorreta → falha

Blockchain Multiusuário

1. Usuário A adiciona bloco → criptografado corretamente
2. Usuário B adiciona bloco → criptografado corretamente
3. Usuário A lê sua blockchain → descriptografa seus blocos, mas não consegue ler blocos de B
4. Tentativa de modificar ciphertext → falha de integridade
5. Alterar hash_prev → erro de validação da cadeia

KDF

1. Mesma senha + mesmo salt → mesma chave
2. Salt diferente → chave diferente

5. Entregáveis

1. Código-fonte deve ser postado no moodle, juntamente com um tutorial de execução da aplicação. Deve ser possível executar a aplicação com os arquivos anexados dentro do código.
2. Documentação explicando:
 - uso de TOTP
 - derivação de chave simétrica
 - criptografia por bloco e encadeamento da blockchain

O código deve ser apresentado e explicado pela equipe. Apresentação desta questão será feita de maneira presencial ou online via Google Meet. Todos os membros da equipe devem apresentar para receber nota. Será usado o próprio código durante a apresentação: a aplicação é executada e depois o código é explicado. A apresentação deve ser agendada na semana seguinte à entrega do trabalho (períodos da tarde e noite).

6. Atenção especial

Atenção especial deve ser dispensada na criação e gerenciamento de parâmetros criptográficos da aplicação:

- i. O sistema **não poderá usar** chave e IV armazenados pelo cliente em variáveis globais ou de ambiente no momento da decifragem. Você deve agir como se o cliente e o servidor estivessem localizados em máquinas diferentes.
- ii. **Para gerar as chaves/IVs devem ser usados o PBKDF2 ou o Scrypt.**
- iii. Deve ser usada **criptografia autenticada para cifragem e decifragem.**
- iv. Decisões próprias sobre formatos e parâmetros devem ser feitas.
- v. **NÃO É PERMITIDO** ter chaves e IV fixos e escritos no próprio código.
- vi. Se forem guardados, os parâmetros devem ser guardados em arquivo cifrado. Apenas o salt pode ficar guardado sem cifragem.

Avisos:

**** Se tiver cópias de código, todos os envolvidos receberão nota zero nesta tarefa.**